Study guide

Voice conversion and voice cloning

From RVC to zero-shot foundation models, with the research frontier as of October 2026

Compiled 18 October 2026 from a conversation with Claude. Covers how RVC works, the three ways to train RVC models, how a model that clones any voice from a short clip is built, and the papers pushing the field forward in 2025 and 2026.

Numbers in square brackets, like [5], point to the reference list at the end. Most frontier results come from recent preprints that use different benchmarks, so compare figures across papers with care.

Contents

How RVC works 4

1.1 What RVC is 4

1.2 The core idea 4

1.3 Content encoder 5

1.4 Retrieval index 5

1.5 Pitch 5

1.6 Generator 5

1.7 Training a voice model 6

1.8 Settings you will meet 6

1.9 Typical cover workflow and its limits 6

1.10 Consent 6

Three ways to train an RVC model 8

2.1 What "from scratch" can mean 8

2.2 Reference points for scale 9

2.3 What stays pretrained no matter what 9

2.4 Running a full from-scratch base 9

2.5 When a custom base is worth it 10

Building a model that clones any voice 11

3.1 Why RVC cannot do this 11

3.2 The architecture 11

3.3 The central problem: the training shortcut 11

3.4 How well it works 12

3.5 The real challenge: scale 12

3.6 Datasets and licenses 12

3.7 A realistic plan 12

3.8 Safeguards 13

The research frontier (October 2026) 15

4.1 Synthetic training pairs 15

4.2 Better timbre perturbation 15

4.3 Real-time and streaming 16

4.4 Singing 16

4.5 Broader trends 16

4.6 Suggested reading order 17

Glossary 19

References 21

How to use this guide

The four parts build on each other. Part 1 explains the RVC pipeline you will meet in tools like the RVC WebUI and Applio. Part 2 covers training and what "from scratch" really involves. Part 3 moves from RVC to zero-shot models that copy any voice without retraining. Part 4 summarizes the papers worth reading now, with a suggested reading order. Each part ends with key takeaways and a few questions to check your understanding. A glossary and full reference list with links close the guide.

Part 1

How RVC works

1.1 What RVC is

RVC stands for Retrieval-based Voice Conversion. It is an open-source voice conversion framework from the RVC-Project, released in 2023 and built on the VITS speech synthesis architecture [1]. RVC converts speech to speech: it takes a recording of someone singing or talking and re-renders it so it sounds like a different person. It does not turn text into speech by itself. Someone still has to perform the words and the melody. RVC only changes who it sounds like.

1.2 The core idea

A voice recording carries three kinds of information. There is *what* is being said (the words and speech sounds), *how* it is delivered (pitch, rhythm, emotion), and *who* is saying it (timbre, the quality that makes a voice recognizable). RVC keeps the first two from the source recording and replaces the third with the target voice. Every stage in Figure 1 serves that split.

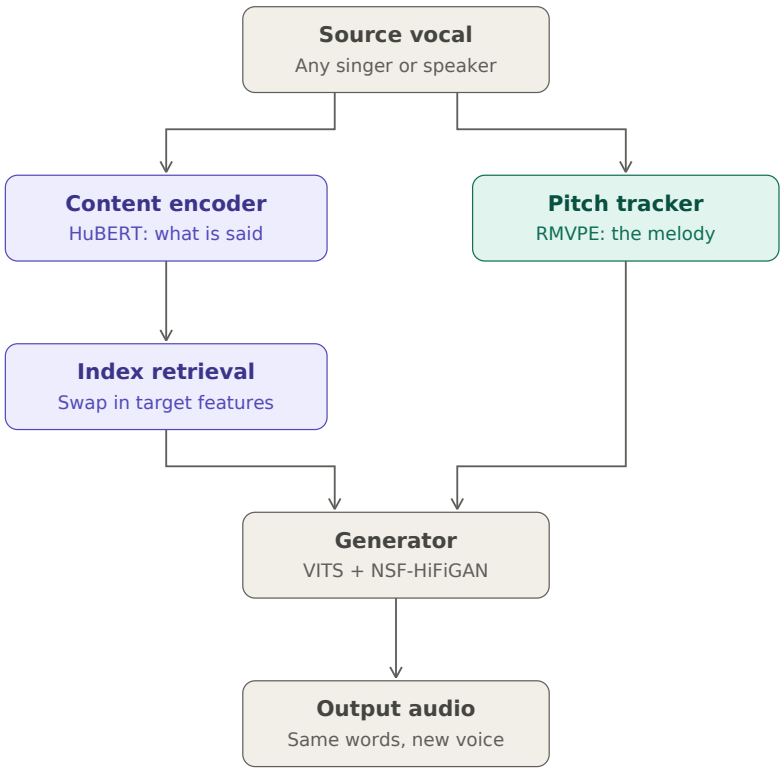


Figure 1. The RVC conversion pipeline. The content path (purple) carries what is said, the pitch path (teal) carries the melody, and the generator renders both in the target voice.

1.3 Content encoder

The first stage on the content path is a pretrained self-supervised speech model called HuBERT [9], or a variant called ContentVec. It slices the audio into short frames, roughly 20 ms each, and turns every frame into a vector of numbers that mostly describes which speech sound is being made: an "ah", an "s", a "t". RVC v1 used a 256-dimensional feature from layer 9 of HuBERT. Version 2 switched to the 768-dimensional output of layer 12 and added three period discriminators for training [1].

These vectors remove most of the original speaker's identity, but not all of it. Some of the source singer's timbre still leaks through. That leakage is the problem the next stage is designed to fix.

1.4 Retrieval index

The retrieval step is where RVC gets its name. When you train a voice model, RVC saves the HuBERT vectors from every frame of the target speaker's training audio into a searchable FAISS index. That is the `.index` file you see distributed alongside models.

During conversion, each frame of the source is matched against that index. The inference code finds the 8 nearest frames the target speaker actually produced, weights them by inverse squared distance so closer matches count more, and averages them. (The README describes the idea more simply as top-1 retrieval [1].) The averaged vector is then blended with the original using the index rate:

```
feature = index_rate × retrieved + (1 − index_rate) × original
```

At a high index rate, the features are built mostly from the target's own recorded sounds, so the voice and accent come out more like the target. Push it too high and articulation can turn mushy, because the nearest match is not always the right phoneme.

1.5 Pitch

Pitch runs on a separate path. A pitch tracker extracts the fundamental frequency (F0), which is the melody, as a curve over time. RMVPE is the usual choice [1], with crepe and harvest as alternatives. Because pitch is handled on its own, you can shift it independently. The transpose setting does this, for example moving a male vocal up 12 semitones for a female target.

1.6 Generator

The generator is a neural network adapted from VITS, with a decoder called NSF-HiFiGAN. It takes the content features, the pitch curve and a speaker embedding, and synthesizes a new waveform in the target's timbre at 32, 40 or 48 kHz. NSF stands for neural source-filter: the decoder first builds an internal tone at the exact pitch, then shapes it into a voice. That is a big reason RVC holds sung notes well, where older converters often wobbled or cracked.

1.7 Training a voice model

You never train an RVC voice from random weights. You fine-tune a base model that has already learned general voice synthesis. The official base was trained on nearly 50 hours of the VCTK dataset [1], which is why 10 to 30 minutes of clean, dry audio of the target voice is usually enough. The README recommends at least 10 minutes of low-noise speech [1].

Training slices the audio into short clips, extracts content features and pitch, and teaches the generator to rebuild the target's audio from them. It trains against a discriminator network that tries to tell real recordings from generated ones. You end up with two files: the `.pth` model weights and the `.index` feature file.

1.8 Settings you will meet

Setting	What it does	Practical note
Index rate	Blends retrieved target features with the original content features.	Higher sounds more like the target in timbre and accent. Too high can slur words.
Transpose	Shifts pitch by a number of semitones.	+12 or −12 moves a full octave, typical for male and female swaps.
Pitch method	Chooses the F0 extraction algorithm.	RMVPE is the common default; crepe and harvest are alternatives.
Protect	Keeps voiceless consonants and breath sounds out of index processing.	0.5 switches it off. Lower values protect more but reduce how much the index applies.
Filter radius	Median filter applied to the pitch curve (harvest).	3 or more smooths jittery pitch.
Volume envelope	Blends the loudness contour of the source with that of the output.	Lower values follow the source's dynamics more closely.

1.9 Typical cover workflow and its limits

For AI song covers, the usual workflow is to separate the vocal from the song with a tool like UVR (the RVC WebUI bundles UVR5 models for this [1]), run the vocal through RVC, and mix it back over the instrumental.

The output can only be as good as the input. Breathing, phrasing, emotion and timing all come from the source singer. Reverb, harmonies or bleed from instruments tend to come out as artifacts, and those are the problems the protect setting and clean vocal separation try to reduce.

1.10 Consent

Cloning a real person's voice without permission is an ethical problem and, in many places, a legal one, especially for public figures. Get consent from anyone whose voice you train on.

Key takeaways

- RVC is speech-to-speech. A performer supplies the words and melody; RVC changes only the voice.
- HuBERT features carry the content. The retrieval index swaps in the target's own features to reduce timbre leakage, controlled by the index rate.
- Pitch is extracted separately (usually RMVPE), so you can transpose freely.
- A VITS generator with an NSF-HiFiGAN decoder renders the waveform. Each model holds exactly one voice.
- Every voice model is a fine-tune of a pretrained base on 10 to 30 minutes of clean audio.

Check your understanding

- If HuBERT already removes most speaker identity, why does RVC need a retrieval step at all?
- What trade-off does the index rate control, and what failure appears at the extreme?
- Why is the pitch path kept separate from the content path?
- Which two files make up a trained RVC voice, and what does each one hold?

Part 2

Three ways to train an RVC model

2.1 What "from scratch" can mean

"Training from scratch" means two very different things in RVC, and the amount of work differs a lot. The first meaning is training your own voice model for one person. That is the normal workflow, and it is what most tutorials mean by "training a model", even though under the hood it is fine-tuning a pretrained base.

The second meaning is training with no pretrained base at all, where the generator and discriminator start from random weights. That produces a new base model, not a voice, and it is a much bigger project. Between the two sits a middle path: building a custom base by continuing to train an existing one on your own multi-speaker data. Figure 2 compares all three.

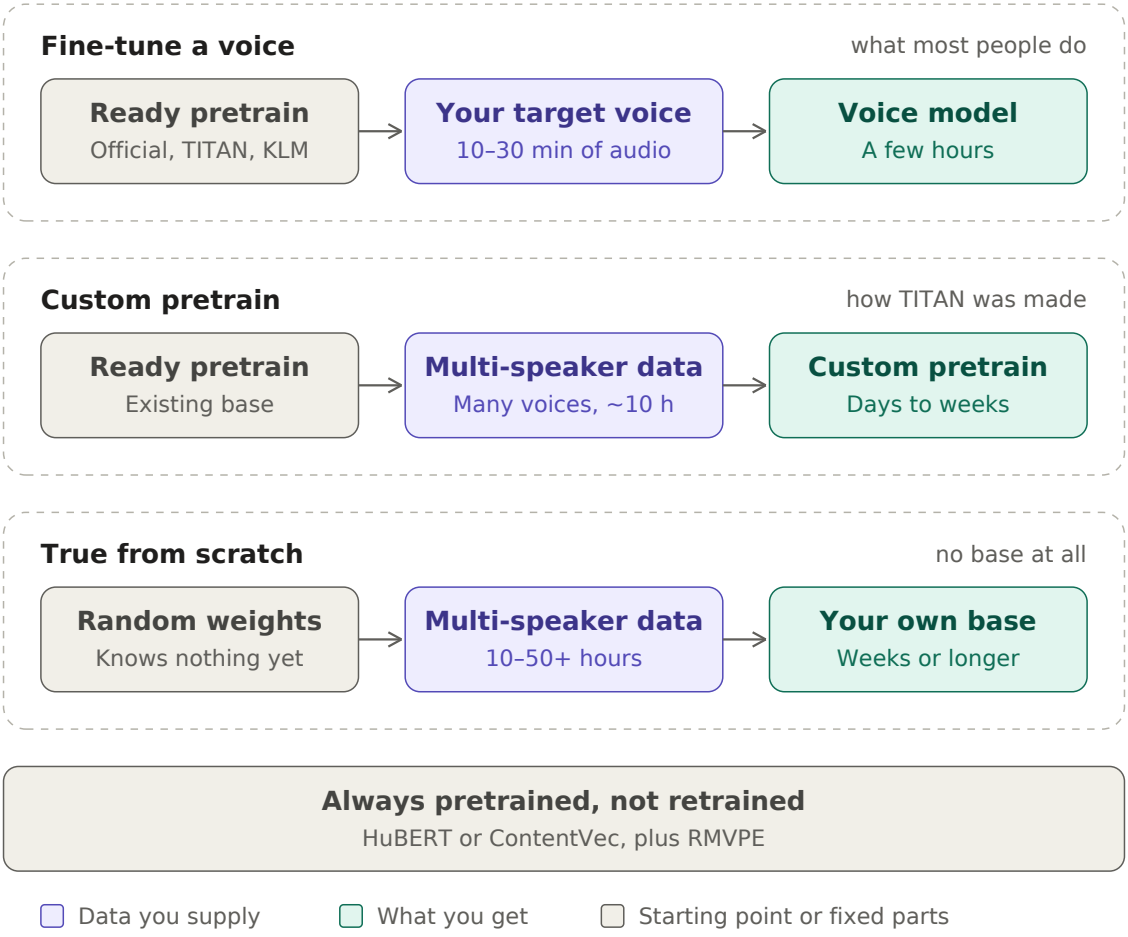


Figure 2. The three training routes. Routes two and three produce a new base model (a G and D file pair), which you then load as a custom pretrain and fine-tune for each voice using route one. Times assume a single consumer GPU.

Route	Starts from	Data you supply	Typical time	Result
Fine-tune a voice	Ready pretrain (official, TITAN, KLM and others)	10–30 min of one voice	A few hours	Voice model (.pth and .index)
Custom pretrain	An existing base	Many speakers, about 10 h	Days to weeks	New base (G and D)
True from scratch	Random weights	Many speakers, 10–50+ h	Weeks or longer	New base (G and D)

2.2 Reference points for scale

- The original RVC base was trained on nearly 50 hours of the VCTK dataset [1].
- The community TITAN-Medium pretrain used an 11.15-hour dataset drawn from Expresso. It was trained in Applio on a single RTX 3060 Ti at batch size 8 for three weeks, and it was itself built on top of the original RVC base rather than started from random weights [3].
- Applio's documentation strongly recommends starting from its pretrained base weights and warns that training from scratch needs far more data and time [2].

2.3 What stays pretrained no matter what

Even a true from-scratch run only covers part of the system. HuBERT (or ContentVec) and RMVPE are separate pretrained models, and you keep using them as they are. You train only the generator, which produces the audio, and the discriminator, the critic network it learns against. Retraining the content encoder is a research-lab project: HuBERT Base learned from 960 hours of LibriSpeech, and the Large model from about 60,000 hours of Libri-Light [9].

2.4 Running a full from-scratch base

Data. A base model has to learn how voices work in general, so it needs many speakers rather than one: dozens of them, male and female, across a wide pitch range, with singing included if you plan to convert singing. Everything should be clean and dry (no music, no reverb) and at the sample rate you will train at. Given the reference points above, treat 10 hours as a floor; more is better.

Settings. In Applio, turn off the Pretrained option. Choose the sample rate, vocoder and embedder carefully before you start, because every voice model you later fine-tune on this base has to match them. Applio's docs warn that mixing these settings across stages leads to failed runs or poor models [2].

Training. Expect days to weeks on a single consumer GPU. Watch the mel loss in TensorBoard, but also listen to checkpoints regularly, because GAN loss curves can look fine while the audio still sounds bad. Keep both the G (generator) and D (discriminator) files from your best checkpoint; you need the pair.

Using it. Applio lets you point training at a custom pretrained generator and discriminator in place of the official ones [2]. Load your G and D there, then fine-tune individual voices on top as usual.

2.5 When a custom base is worth it

Building your own base makes sense if your voices are in a language or style the existing bases cover poorly (most are trained heavily on English speech), if you want a sample rate and vocoder combination nobody has released a pretrain for, if you need to know exactly what data is in your base for licensing reasons, or if you are doing it to learn.

Otherwise, try existing community pretrains first. Collections on Hugging Face include TITAN, KLM, Ov2Super and several others [4]. The middle path, fine-tuning one of them on your own multi-speaker dataset, is how TITAN was made, and it gives you a custom base for much less compute.

Key takeaways

- Almost everyone who "trains an RVC model" is fine-tuning a pretrained base on one voice.
- A true from-scratch base needs many speakers, at least around 10 hours of clean audio, and weeks of GPU time.
- HuBERT/ContentVec and RMVPE stay pretrained in every route. Only the generator and discriminator are trained.
- Lock sample rate, vocoder and embedder before building a base; later fine-tunes must match.
- A custom pretrain built on an existing base is usually the best value if you need something specialized.

Check your understanding

- Why does a base model need many speakers rather than many hours of one speaker?
- Which two files make up a pretrain, and why must you keep both?
- Name two situations where building your own base is worth the cost.

Part 3

Building a model that clones any voice

3.1 Why RVC cannot do this

An RVC model cannot become a "clone any voice" model, even trained at much larger scale. RVC stores one voice in its weights, so every new voice needs its own training run. A model that clones any voice works differently: it reads the target voice from a short reference clip each time you use it, with no retraining. This is called zero-shot voice conversion. When the input is text instead of speech, it is called zero-shot text-to-speech; open examples include XTTS v2, F5-TTS and CosyVoice. This guide focuses on voice conversion, which is the natural next step from RVC.

3.2 The architecture

Figure 3 shows the design most current open systems use, modeled on Seed-VC [5]. Seed-VC uses OpenAI's Whisper as the content encoder and NVIDIA's BigVGAN (or HIFT) as the vocoder, so the diffusion model in the middle is the only part you build. Its reference clip can be anywhere from about 1 to 30 seconds.

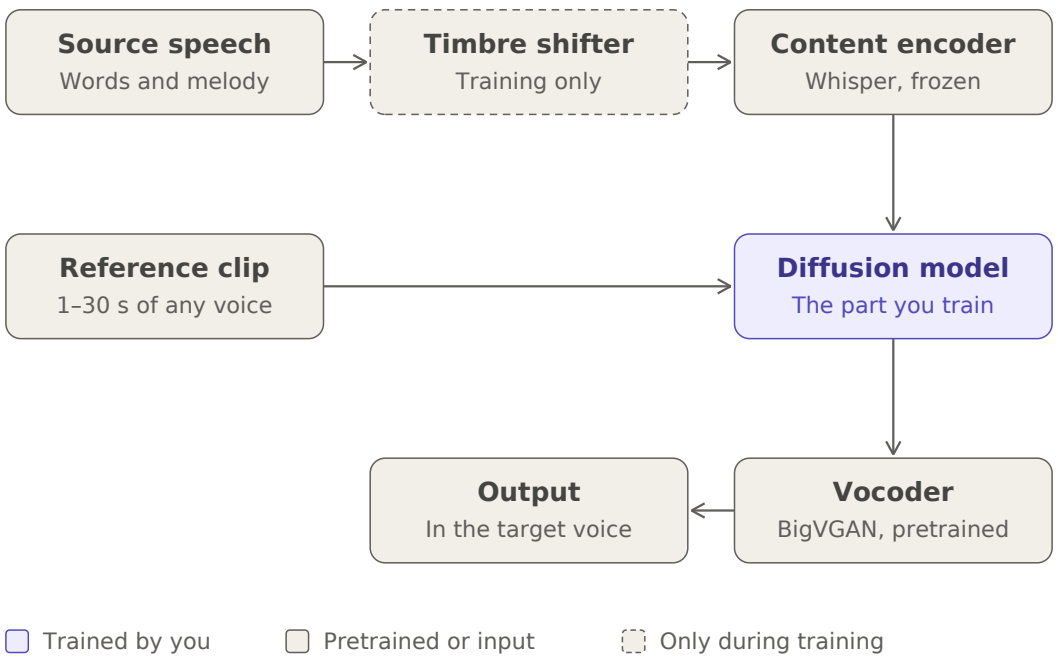


Figure 3. A zero-shot voice conversion system in the style of Seed-VC. Only the diffusion model is trained. The timbre shifter is used during training and removed at inference.

3.3 The central problem: the training shortcut

The hard part is training. You almost never have the same sentence recorded in two different voices, so the model learns by rebuilding a clip from its own content features plus a different clip of the same speaker. That creates a shortcut. Some of the speaker's timbre leaks through the content features, and the model learns to copy it instead of reading the

reference. At inference, when the speakers really are different, it then fails.

Seed-VC fixes this by running the source through an external timbre shifter during training, which reduces leakage and makes training look more like real use. Its diffusion transformer also reads the whole reference clip as context to pick up fine timbre detail [5]. Almost every paper in Part 4 is a different attack on this same shortcut.

3.4 How well it works

The approach works well. With one random utterance as the prompt and no training on the target singers, Seed-VC beat speaker-specific RVCv2 models on speaker similarity and intelligibility. Its audio quality, measured by DNSMOS, scored slightly below RVCv2 [6]. In other words, zero-shot models now match or beat per-voice models on sounding like the target, while dedicated models still hold a small edge in raw audio quality.

3.5 The real challenge: scale

A model can only generalize to voices it has never heard if it trained on a huge variety of speakers. The number of distinct speakers matters much more than the total hours from any one voice. For reference, the REF-VC paper trained on Emilia, roughly 100,000 hours of speech. Its diffusion transformer had 12 layers and about 100 million parameters, and trained for 1 million steps on 8 A100 GPUs [7]. That is well beyond a single consumer GPU.

3.6 Datasets and licenses

Read the license before you download anything. The original Emilia is restricted to non-commercial use, while the newer Emilia-YODAS portion is under a permissive license [8].

Dataset	Size	Languages	License	Notes
LibriTTS	585 h, about 2,400 speakers	English	CC BY 4.0	Audiobook speech; a good starter set [10].
Emilia	Over 101k h	EN, ZH, DE, FR, JA, KO	CC BY-NC 4.0	Non-commercial only. In-the-wild speech from podcasts and video [8].
Emilia-YODAS	Over 114k h	Same six	CC BY 4.0	Commercial use allowed with attribution [8].
Emilia-Large	Over 216k h	Same six	Mixed	Emilia plus Emilia-YODAS combined [8].
YODAS2	About 500k h	100+ languages	CC BY 3.0 sources	Raw YouTube speech; you must clean it yourself [8].

3.7 A realistic plan

Stage 1: reproduce. Take an existing open system such as Seed-VC and train it on a small dataset like LibriTTS, which fits on one GPU and teaches you the whole pipeline. Zero-VC (Part 4) was trained only on LibriTTS, about 460 hours after filtering, which shows how far a modest dataset can go [14].

Stage 2: scale. Move to a large multi-speaker dataset (Emilia-YODAS if you need a permissive license) and rent multi-GPU compute for the full run.

Stage 3: measure. Evaluate the way the papers do, using the metrics in the table below.

There is also a middle path, like the custom pretrain route for RVC: start from released weights and adapt them to your language or domain. Twinkle-VC did this, adapting and extending Seed-VC for a Vietnamese voice conversion challenge [19].

Metric	What it measures	How it is computed
Speaker similarity (S-SIM, SS-R)	Does the output sound like the target?	Cosine similarity between speaker embeddings of output and reference, often from WavLM-Large tuned for speaker verification.
Source leakage (SS-S)	Does the source voice still come through?	The same similarity, measured against the source. Lower is better.
WER / CER	Are the words preserved?	Transcribe the output with a recognizer such as Whisper-large-v3 and compare with the reference text.
F0 Pearson (FPC)	Is the melody and prosody kept?	Correlation between source and output pitch curves.
DNSMOS / UTMOS	Naturalness and audio quality	Neural networks that predict human quality ratings.
N-MOS / S-MOS	Human judgement	Listeners rate naturalness and similarity on a 1–5 scale.

These are the metrics used in the MimicLM and Zero-VC papers [11][14], so learning them lets you read the result tables in Part 4.

3.8 Safeguards

A model that clones anyone from a few seconds of audio is exactly what phone scammers use. If you release one, add an inaudible watermark to its output (Meta's AudioSeal is open source [21]), use only properly licensed training data, add consent or speaker-verification checks before cloning a voice, and document what the model is for. The MimicLM authors make the same recommendations, adding synthetic-speech detection to the list [11].

Key takeaways

- Zero-shot models read the target voice from a short reference clip at inference time; nothing is retrained per voice.
- The core training difficulty is the shortcut: timbre leaks through content features when source and target are the same speaker.
- Seed-VC's fix is a timbre shifter during training plus a diffusion transformer that reads the whole reference clip.
- Generalizing to unseen voices requires a huge number of distinct speakers: think tens of thousands of hours.
- Check dataset licenses early. Emilia is non-commercial; Emilia-YODAS is permissive.

Check your understanding

- Explain in your own words why training on same-speaker pairs creates a shortcut.
- Why is speaker count more important than hours per speaker for a zero-shot model?
- What is the difference between SS-R and SS-S, and why report both?

Part 4

The research frontier (October 2026)

Nearly every recent paper works on the problem from section 3.3: there are almost no recordings of the same sentence in two different voices, so models pick up shortcuts during training. The papers worth reading fall into three groups, plus a set of broader trends.

4.1 Synthetic training pairs

The biggest shift is generating synthetic training pairs instead of relying on disentanglement tricks.

MimicLM (April 2026) is the clearest example [11]. It takes a real utterance from speaker A, uses a TTS model (CosyVoice 2) to say the same words in another voice, B, and then trains the model to turn that synthetic clip back into the original real recording, using another real clip of A as the reference. The authors call this role-swapping. Because the training targets are real speech, the model's quality is not capped by the TTS system that generated the pairs.

The model itself is a decoder-only language model built on Qwen2.5-0.5B, extended with speech tokens from CosyVoice 2's tokenizer (25 tokens per second) and trained to predict text and audio tokens interleaved, which helps preserve the words. They built about 8.5 million English training triplets (roughly 18,000 hours) from Emilia, after an ASR check removed a third of the generated pairs. Training on synthetic inputs creates a gap with real inputs at inference, so they added DPO preference training, which cut the error rate on real-to-real conversion from 15.8% to 13.8% [11].

System	Naturalness (N-MOS)	Similarity (S-MOS)	Objective S-SIM
SeedVC v2	3.14	3.03	0.553
Vevo	3.85	4.32	0.652
MimicLM (with DPO)	4.71	4.62	0.601

Table 4.1. MimicLM against the strongest voice imitation baselines [11]. Human raters preferred MimicLM, but on the automatic speaker-similarity metric it still trails Vevo.

From A to B to A (June 2026), a "palindromic" approach, works along similar lines. It uses controlled synthetic data generation to get supervision without aligned speech, and reports better speaker similarity than recent systems while staying comparable on word error rate and quality scores [12].

4.2 Better timbre perturbation

Zero-VC (June 2026) improves on Seed-VC's perturbation idea [14]. It replaces the timbre shifter with an off-the-shelf speaker anonymization model, a system designed to hide who is speaking while keeping the words and prosody intact. A WavLM-Large encoder turns the reference into a speaker embedding, and a causal HiFi-GAN-style decoder generates audio frame by frame.

Perturbation method	Similarity to source (lower is better)	Pitch kept (FPC)
LSCodec-style signal processing	0.704	0.891
Seed-VC's timbre shifter	0.411	0.688
Speaker anonymization (Zero-VC)	0.119	0.718

Table 4.2. How much of the source voice survives each perturbation method, measured on the perturbed audio itself [14].

The cleaner features let the model work without looking ahead in the audio, bringing algorithmic latency down to a single 20 ms frame, compared with 40 to 60 ms for earlier streaming systems. It is also the most approachable paper for a solo builder: it was trained only on LibriTTS (about 460 hours after filtering) and runs faster than real time on a CPU, with a real-time factor of 0.063 [14].

4.3 Real-time and streaming

X-VC (April 2026) converts voices in a single step inside the latent space of a pretrained neural codec [13]. It trains on generated paired data using standard, reconstruction and reversed modes, and streams with chunked inference and overlap smoothing. It reports the best streaming word error rate in both English and Chinese among the systems it compares, and its code and checkpoints are public, which makes it a good system to run and study.

MeanVC (October 2025) targets lightweight streaming conversion using mean flows, a faster variant of flow matching [17]. MimicLM includes it as a baseline in its comparisons [11].

4.4 Singing

Singing is where RVC is most used, and two papers stand out.

Vevo2 handles speech and singing in one model [15]. It introduces a prosody tokenizer that captures melody without needing musical notation, and a 12.5 Hz content-style tokenizer that keeps words, prosody and style while separating out timbre. An autoregressive stage built on Qwen2.5-0.5B predicts content-style tokens, and a flow-matching stage controls timbre. The authors report that training on speech and singing together helps both. The weights are on Hugging Face.

HQ-SVC is smaller in scope [16]. It aims for high-quality zero-shot singing conversion with limited resources, training on a single consumer GPU with small datasets, and it can also upsample voice audio (voice super-resolution).

4.5 Broader trends

- **Voice conversion is turning into speech language modeling.** MimicLM and Vevo2 both start from a small text LLM (Qwen2.5-0.5B), add speech tokens to its vocabulary, and train it to generate audio.
- **Techniques are borrowed from LLM training.** Synthetic data pipelines and preference alignment with DPO now appear in voice conversion papers.

- **"Voice imitation" goes beyond timbre.** Recent systems also copy accent and emotion. That is harder, and these systems usually have higher word error rates than timbre-only conversion [11].
- **Latency is a first-class goal.** Codec-space one-step conversion (X-VC) and strictly causal models (Zero-VC) push toward real-time use in calls and streams.

4.6 Suggested reading order

For a foundation-model goal, read them in this order. Start with Zero-VC, since its scale is closest to what one person can reproduce. Then X-VC, because you can run the released code. Then MimicLM, for the recipe to build training data at scale. Read Vevo2 and HQ-SVC if singing matters to you. Keep Seed-VC and Vevo as background, since almost every paper compares against them.

Paper	arXiv date	Core idea	Released
Seed-VC [5]	Nov 2024	Timbre shifter plus diffusion transformer reading the whole reference	Code and weights
Vevo [18]	Feb 2025	Voice imitation through self-supervised disentanglement (ICLR 2025)	In the Amphion toolkit [20]
Vevo2 [15]	Aug 2025	One model for speech and singing; notation-free prosody tokenizer	Weights on Hugging Face
MeanVC [17]	Oct 2025	Lightweight streaming conversion with mean flows	See paper
HQ-SVC [16]	Nov 2025	Zero-shot singing conversion on a single consumer GPU	See paper
MimicLM [11]	Apr 2026	Role-swapped synthetic sources with real targets, plus DPO	Demo page
X-VC [13]	Apr 2026	One-step conversion in codec latent space for streaming	Code and checkpoints
A to B to A [12]	Jun 2026	Synthetic supervision without aligned speech	See paper
Zero-VC [14]	Jun 2026	Speaker anonymization as perturbation; 20 ms latency	Demo samples

Key takeaways

- The frontier is about solving the missing-parallel-data problem, either with synthetic pairs (MimicLM, X-VC, A to B to A) or with better perturbation (Zero-VC).
- Using real recordings as training targets avoids being capped by the quality of the TTS that made the pairs.
- Speaker anonymization removes far more source timbre than earlier perturbation methods and enables 20 ms latency.
- Small LLM backbones with speech tokens are becoming a standard architecture for voice generation.

Check your understanding

- In MimicLM's role-swapping, which clip is synthetic, which is the reference and which is the target? Why does that order matter?
- Why does a cleaner perturbation let Zero-VC drop its lookahead?
- What does DPO fix in MimicLM, and how are the preference pairs chosen?

Reference

Glossary

Term	Meaning
BigVGAN	A neural vocoder from NVIDIA that turns mel spectrograms into high-quality waveforms.
CER / WER	Character or word error rate: how often a speech recognizer mishears the output. Measures intelligibility.
ContentVec	A variant of HuBERT trained to carry less speaker information, used as an RVC embedder.
Diffusion model	A generator that learns to turn noise into data step by step. Seed-VC uses a diffusion transformer (DiT).
Disentanglement	Separating content, prosody and timbre into different representations so one can be changed without the others.
DNSMOS / UTMOS	Neural predictors of human quality ratings, used as automatic naturalness scores.
DPO	Direct Preference Optimization: training a model to prefer better outputs over worse ones, borrowed from LLM alignment.
F0	Fundamental frequency; the pitch or melody of a voice over time.
FAISS	A library for fast nearest-neighbour search, used for RVC's .index file.
Fine-tuning	Continuing to train a pretrained model on new, usually smaller, data.
Flow matching	A generative training method related to diffusion that often needs fewer sampling steps.
G and D files	Generator and discriminator weights. A pretrain is always a G and D pair.
HiFi-GAN	A GAN-based vocoder architecture; RVC uses an NSF variant of it.
HuBERT	A self-supervised speech model whose internal features mostly describe speech sounds.
Index rate	RVC setting that blends retrieved target features with the original content features.
Mel spectrogram	A time-frequency picture of audio scaled to human hearing; a common intermediate format.
NSF	Neural source-filter: a decoder design that builds a tone at the right pitch, then shapes it into a voice.
Pretrain (base model)	A model already trained on many voices, used as the starting point for fine-tuning.
Pseudo-parallel data	Synthetic pairs of the same sentence in two voices, created to stand in for real parallel recordings.
RMVPE	A pitch extraction model designed to work on vocals, the default in RVC.
Speaker anonymization	Processing that hides who is speaking while keeping words and prosody.
Streaming / lookahead	Processing audio as it arrives. Lookahead is how much future audio the model must wait for.
Timbre	The tone quality that makes a voice recognizable, independent of pitch and words.
Timbre leakage	Source speaker identity surviving into features that should carry only content.

Term	Meaning
UVR	Ultimate Vocal Remover, a tool for separating vocals from instrumentals.
VITS	An end-to-end speech synthesis architecture that RVC's generator is based on.
Vocoder	The final stage that converts an acoustic representation into an audible waveform.
Voice imitation	Copying timbre plus speaking style (accent, emotion), not just timbre.
Watermarking	Embedding an inaudible signal in generated audio so it can later be identified.
Whisper	OpenAI's speech recognition model; its encoder is used as a content encoder in Seed-VC.
Zero-shot	Working on voices never seen in training, from a short reference clip, with no retraining.

References

[1] RVC-Project. Retrieval-based-Voice-Conversion-WebUI, README (English). github.com/RVC-Project/Retrieval-based-Voice-Conversion-WebUI

[2] Applio documentation, training reference. docs.applio.org and mintlify.wiki/IAHispano/Applio/cli/train

[3] blaise-tk. TITAN: a pretrained model for RVC training (model card). huggingface.co/blaise-tk/TITAN

[4] Politrees. RVC_resources: collection of RVC pretrains and HuBERT models. huggingface.co/Politrees/RVC_resources

[5] Liu, S. (2024). Zero-shot voice conversion with diffusion transformers (Seed-VC). arXiv:2411.09943. Code: github.com/Plachtaa/seed-vc

[6] Seed-VC evaluation results (EVAL.md), in the Seed-VC repository. github.com/Plachtaa/seed-vc

[7] REF-VC: robust, expressive and fast zero-shot voice conversion with diffusion transformers. arXiv:2508.04996

[8] He, H. et al. Emilia: a large-scale, extensive, multilingual, and diverse dataset for speech generation. arXiv:2501.15907. Dataset: huggingface.co/datasets/amphion/Emilia-Dataset

[9] Hsu, W.-N. et al. (2021). HuBERT: self-supervised speech representation learning by masked prediction of hidden units. arXiv:2106.07447

[10] Zen, H. et al. (2019). LibriTTS: a corpus derived from LibriSpeech for text-to-speech. arXiv:1904.02882

[11] Feng, T. et al. (2026). MimicLM: zero-shot voice imitation through autoregressive modeling of pseudo-parallel speech corpora. arXiv:2604.11552

[12] From A to B to A: palindromic zero-shot voice conversion with non-parallel data (2026). arXiv:2606.08843

[13] Zheng, Q. et al. (2026). X-VC: zero-shot streaming voice conversion in codec space. arXiv:2604.12456. Code: github.com/Jerrister/X-VC

[14] Li, Y. et al. (2026). Zero-VC: zero-lookahead streaming voice conversion via speaker anonymization. arXiv:2606.20218. Demo: amphionteam.github.io/Zero-VC-demo

[15] Zhang, X. et al. Vevo2: bridging controllable speech and singing voice generation via unified prosody learning. arXiv:2508.16332. Weights: huggingface.co/RMSnow/Vevo2

[16] HQ-SVC: towards high-quality zero-shot singing voice conversion in low-resource scenarios. arXiv:2511.08496

[17] Ma, G. et al. (2025). MeanVC: lightweight and streaming zero-shot voice conversion via mean flows. arXiv:2510.08392

[18] Zhang, X. et al. (2025). Vevo: controllable zero-shot voice imitation with self-supervised disentanglement. ICLR 2025. arXiv:2502.07243

[19] Twinkle-VC: a robust and high-quality zero-shot voice conversion. VLSP 2025 voice conversion challenge.

[20] Amphion: an open-source audio, music and speech generation toolkit. github.com/open-mmlab/Amphion

[21] Meta. AudioSeal: localized audio watermarking. github.com/facebookresearch/audioseal